

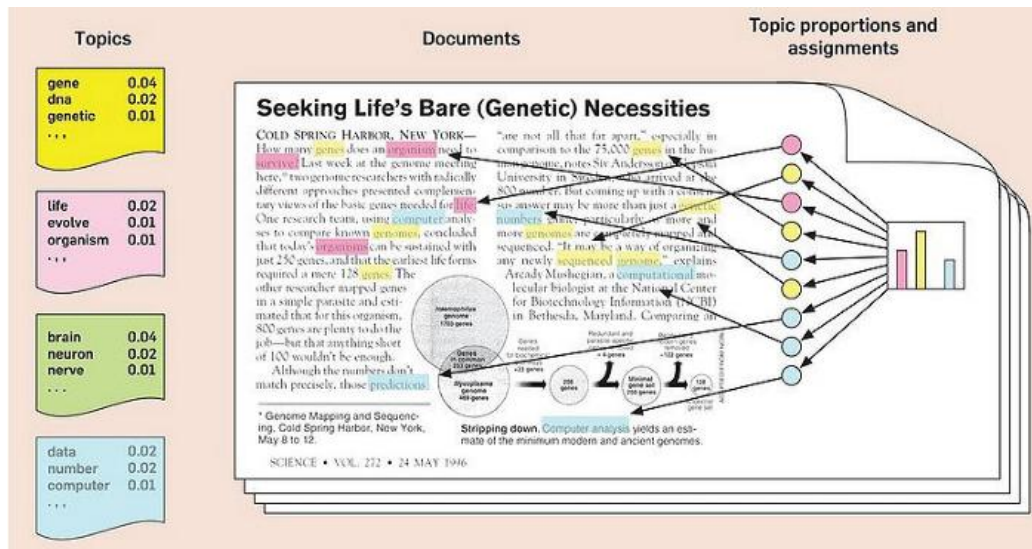
Topic Modelling

- Topic modelling in NLP, which assigns topic to a given corpus based on the words present.
- Topic modelling is important, because in this world full of data it has become increasingly important to categorise the documents.
- Topics can be thought of as *keywords* which can describe a document, for example, for a topic sports the words that come to our mind are volleyball, basketball, tennis, cricket etc.
- A **topic model** is a model, which can automatically detect topics based on the words appearing in a document.
- It is important to note that topic modelling is different to topic classification.
 - Topic classification is a supervised learning while topic modelling is a unsupervised learning algorithm.

Latent Dirichlet Allocation

LDA, short for Latent Dirichlet Allocation is a technique used for topic modelling.

- **Latent** means hidden, something that is yet to be found.
- **Dirichlet** indicates that the model *assumes* that the topics in the documents and the words in those topics follow a Dirichlet distribution.
- **Allocation** means to giving something, which in this case are topics.



The general steps in the LDA are as follows:

Hyperparameters in LDA:

There are three hyperparameters in LDA

- α hyperparameter controls the number of topic *expected* in the document.
- β hyperparameter controls the distribution of words per topic in the document, and
- K defines how many topics we need to extract.

LDA

LDA is used to classify text in a document to a particular topic.

It builds a topic per document model and words per topic model, modelled as Dirichlet distributions.

- Each document is modelled as a multinomial distribution of topics and each topic is modelled as a multinomial distribution of words.
- LDA assumes that every chunk of text we feed into it will contain words that are somehow related. Therefore, choosing the right corpus of data is crucial.
- It also assumes documents are produced from a mixture of topics. Those topics then generate words based on their probability distribution.

We have the following 3 documents:

1. **Doc 1:** "I like to eat broccoli and bananas."
2. **Doc 2:** "I ate a banana and spinach smoothie."
3. **Doc 3:** "Cherries are my favorite fruit."

Vocabulary

From the documents, extract a vocabulary (unique words):

[broccoli, bananas, eat, like, to, and, I, ate, a, spinach, smoothie, cherries, are, my, favorite, fruit]

This gives us a vocabulary size $V=16$

Input Matrix (Document-Word Matrix)

Create a document-word matrix $D * V$, here D is the number of documents.

	broccoli	bananas	eat	like	to	and	I	ate	a	spinach	smoothie	cherries	are	my	favorite	fruit
Doc 1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
Doc 2	0	1	0	0	0	1	1	1	1	1	1	0	0	0	0	0
Doc 3	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1

Step 1: Assign Initial Topics

Assume we want 2 topics $K=2$.

Randomly assign each word in each document to one of the two topics.

For example:

- Topic 1: Related to **food/recipes**.
- Topic 2: Related to **favorites/fruits**.

Initial topic assignments might look like this:

Word	Doc 1 Topic	Doc 2 Topic	Doc 3 Topic
broccoli	1	-	-
bananas	1	1	-
eat	1	-	-
like	1	-	-
to	1	-	-
and	1	1	-
I	1	1	-
ate	-	1	-
a	-	1	-
spinach	-	1	-
smoothie	-	1	-
cherries	-	-	2
are	-	-	2
my	-	-	2
favorite	-	-	2
fruit	-	-	2

Step 2: Count Words by Topic

Count the occurrences of words for each topic in each document.

Here's an example of the counts for **Topic 1** and **Topic 2**.

	Topic 1 Count	Topic 2 Count
broccoli	1	0
bananas	2	0
eat	1	0
like	1	0
to	1	0
and	2	0
I	2	0
ate	1	0
a	1	0
spinach	1	0
smoothie	1	0
cherries	0	1
are	0	1
my	0	1
favorite	0	1
fruit	0	1

Step 3: Update Topic Probabilities

For each word, update its topic assignment based on the **probability distribution**:

$$P(\text{topic } k | \text{word, document}) \propto P(\text{word} | \text{topic } k) \times P(\text{topic } k | \text{document})$$

Example Calculation

For word "bananas" in Doc 1:

- $P(\text{"bananas"} | \text{Topic 1}) = \frac{\text{Count of "bananas" in Topic 1} + \beta}{\text{Total words in Topic 1} + V \cdot \beta}$
- $P(\text{"bananas"} | \text{Topic 2}) = \frac{\text{Count of "bananas" in Topic 2} + \beta}{\text{Total words in Topic 2} + V \cdot \beta}$
- $P(\text{Topic 1} | \text{Doc 1}) = \frac{\text{Count of words assigned to Topic 1 in Doc 1} + \alpha}{\text{Total words in Doc 1} + K \cdot \alpha}$

Where α and β are hyperparameters controlling the Dirichlet priors.

Step 4: Iterative Updates

Repeat the assignment and probability updates for a set number of iterations until convergence.

Output

The result will be two topics, with words distributed across them:

- **Topic 1:** [broccoli, bananas, eat, like, and, spinach, smoothie]
- **Topic 2:** [cherries, are, my, favorite, fruit]

This indicates that **Topic 1** relates to food/recipes, and **Topic 2** relates to favorite fruits.

Non-Negative Matrix Factorization (NMF) is a popular dimensionality reduction technique often used for **topic modeling** in Natural Language Processing (NLP).

It decomposes a term-document matrix into two smaller matrices: one representing the topics and the other representing the weights of those topics in the documents.

Concept of NMF

Given a **term-document matrix** V , where each row represents a term (word) and each column represents a document, NMF tries to find two non-negative matrices W and H such that:

$$V \approx W \cdot H$$

- V : Original term-document matrix of size $m \times n$ (terms $\times$ documents).
- W : Term-topic matrix of size $m \times k$, where k is the number of topics. Each column represents the terms associated with a topic.
- H : Topic-document matrix of size $k \times n$, where each row represents the topic distribution for a document.

Here:

- **Non-negativity** ensures that the representation is interpretable (e.g., no negative weights).
- $W[i, j]$: The importance of term i in topic j .
- $H[i, j]$: The importance of topic i in document j .